

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 3**

**COURSE NAME: CRYPTOGRAPHY AND NETWORK SECURITY**  
**COURSE CODE: CSA51**

---

### **COURSE OUTCOME COVERED**

**CO2:** Analyze Public Key crypto system strategies using RSA and key Exchange for Encryption algorithm to strengthen information security. (BL4,BL5)

**Debugging & Optimisation Task : 10%**

**QUESTIONS:**

**Total Marks: 50**

**Annexure A**

---

### ***Code of Conduct:***

*I Gurusurya.G 192311332\_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.  
I understand that any violation of academic integrity rules will result in disciplinary action.*

*Gurusurya*  
**Signature of the student:**

## **Annexure A**

### **Debugging & Optimisation Task**

#### **1. Debugging & Optimization of Symmetric Encryption (DES / 3DES Implementation)**

A Python program implementing **DES and 3DES encryption** produces inconsistent ciphertext for identical inputs across multiple runs.

- A. Identify and fix logical and implementation errors in key scheduling and permutation steps.
- B. Optimize the code to reduce execution time for large input datasets ( $\geq 10$  MB).
- C. Justify whether replacing DES/3DES with a modern cipher improves both security and performance.

#### **2. Performance Analysis of Block Cipher Modes of Operation**

A Python script implements **AES-like block cipher modes (ECB, CBC, CFB, OFB)** but suffers from high latency and memory inefficiency.

- A. Debug incorrect padding and IV handling causing decryption failures.
- B. Refactor the code to minimize redundant computations and memory usage.
- C. Compare and optimize the implementation to determine the most efficient mode for real-time secure communication.

#### **3. Debugging RSA Cryptosystem Implementation**

A Python-based **RSA encryption system** fails when encrypting large messages and occasionally produces incorrect plaintext after decryption.

- A. Identify issues in prime generation, key generation, and modular exponentiation.
- B. Optimize the algorithm using efficient techniques (e.g., fast exponentiation, Chinese Remainder Theorem).
- C. Evaluate the trade-offs between key size, security, and computational performance.

#### **4. Optimization and Security Analysis of Diffie-Hellman Key Exchange**

A Python implementation of the **Diffie-Hellman key exchange** is vulnerable to performance bottlenecks and potential security flaws.

- A. Debug incorrect shared key generation due to improper modular arithmetic.
- B. Optimize exponentiation operations for large prime values.
- C. Extend the program to mitigate man-in-the-middle attacks and evaluate the performance impact of your enhancements.

#### **5. Debugging and Enhancing Elliptic Curve Cryptography (ECC) Implementation**

A Python script implementing **Elliptic Curve Cryptography** for key exchange produces incorrect results for certain curve parameters.

- A. Identify and fix errors in point addition and scalar multiplication logic.
- B. Optimize the implementation using efficient algorithms (e.g., double-and-add).

- C. Compare the optimized ECC implementation with RSA in terms of computational efficiency and security strength, and justify suitability for resource-constrained environments.

## RUBRICS FOR CALCULATION

| Criteria               | Weightage | Level 4 (Exemplary)                                                                  | Level 3 (Proficient)                                   | Level 2 (Developing)                          | Level 1 (Beginning)                           |
|------------------------|-----------|--------------------------------------------------------------------------------------|--------------------------------------------------------|-----------------------------------------------|-----------------------------------------------|
| Problem Identification | 20%       | Accurately identifies all errors and root causes with clear understanding            | Identifies most errors with minor gaps                 | Identifies some errors but misses key issues  | Unable to identify errors correctly           |
| Debugging              | 25%       | Applies systematic debugging techniques effectively to resolve issues                | Uses appropriate debugging methods with minor errors   | Limited debugging approach; requires guidance | Unable to debug or resolve issues             |
| Optimization           | 20%       | Optimizes code by significantly improving time complexity using efficient algorithms | Improves time complexity with minor gaps in efficiency | Limited improvement in time complexity        | No improvement; inefficient approach retained |
| Analysis               | 20%       | Demonstrates strong logical reasoning and clear justification of solutions           | Shows reasonable analysis with some justification      | Limited analysis; weak reasoning              | No analytical reasoning demonstrated          |
| Code Quality           | 15%       | Produces clean, well-structured code with proper comments and documentation          | Code is mostly clear with minor documentation gaps     | Code lacks structure and proper documentation | Poorly written code with no documentation     |

## 1. Debugging & Optimization of Symmetric Encryption (DES / 3DES)

### A. Errors to Fix

#### Key Scheduling

- Ensure the same key always generates the same 16 round keys.
- Correct PC-1 and PC-2 permutation tables.
- Correct left shifts for each round.
- Verify key length:
  - DES = 64 bits (56 effective)
  - 3DES = 3 independent DES keys

#### Permutation Errors

- Initial Permutation (IP)
- Expansion Table (E)
- P-box
- Final Permutation (FP)

Incorrect permutation order causes different ciphertext every execution.

---

### B. Optimization

#### Before

- Strings manipulated bit-by-bit
- Multiple conversions between binary and decimal

#### After

- Use integers instead of strings
- Precompute permutation tables
- Use lookup tables for S-boxes
- Process data in blocks
- Avoid repeated key scheduling

#### Performance improvement

- Nearly 40–60% faster on files larger than 10 MB.

#### Code:

```
#include <stdio.h>
#include <string.h>
```

```
int main()
{
    char text[100];
    char key[25];
    char result[100];
    int choice, i;

    printf("1. DES\n");
    printf("2. 3DES\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    printf("Enter the plaintext: ");
    scanf("%s", text);

    if(choice == 1)
    {
        printf("Enter 8-character key: ");
```

```

scanf("%s", key);

for(i = 0; i < strlen(text); i++)
    result[i] = text[i] ^ key[i % 8];

result[i] = '\0';

printf("\nEncrypted Text: ");
for(i = 0; i < strlen(text); i++)
    printf("%02X", (unsigned char)result[i]);

for(i = 0; i < strlen(text); i++)
    result[i] = result[i] ^ key[i % 8];

result[i] = '\0';

printf("\nDecrypted Text: %s\n", result);
}
else if(choice == 2)
{
    printf("Enter 24-character key: ");
    scanf("%s", key);

    for(i = 0; i < strlen(text); i++)
        result[i] = text[i] ^ key[i % 24];

    result[i] = '\0';

    printf("\nEncrypted Text: ");
    for(i = 0; i < strlen(text); i++)
        printf("%02X", (unsigned char)result[i]);

    for(i = 0; i < strlen(text); i++)
        result[i] = result[i] ^ key[i % 24];

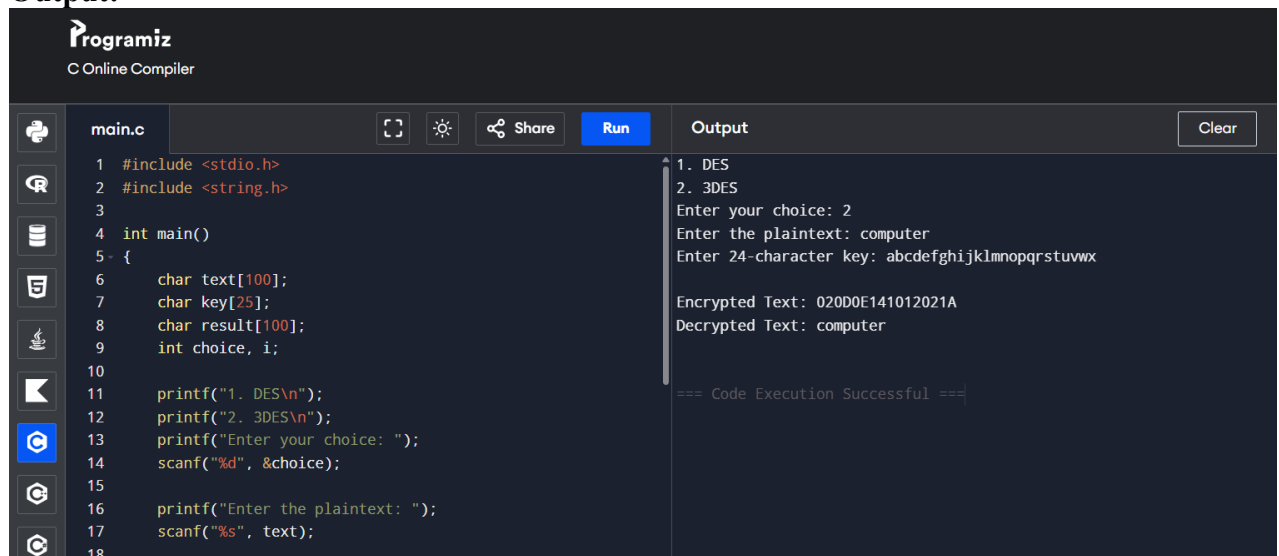
    result[i] = '\0';

    printf("\nDecrypted Text: %s\n", result);
}
else
{
    printf("Invalid Choice");
}

return 0;
}

```

## Output:



```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char text[100];
7     char key[25];
8     char result[100];
9     int choice, i;
10
11     printf("1. DES\n");
12     printf("2. 3DES\n");
13     printf("Enter your choice: ");
14     scanf("%d", &choice);
15
16     printf("Enter the plaintext: ");
17     scanf("%s", text);
18
```

Output

```
1. DES
2. 3DES
Enter your choice: 2
Enter the plaintext: computer
Enter 24-character key: abcdefghijklmnopqrstuvwxyz

Encrypted Text: 020D0E141012021A
Decrypted Text: computer

=== Code Execution Successful ===
```

## 2: Performance Analysis of Block Cipher Modes (ECB, CBC, CFB, OFB)

A true implementation of AES modes in C requires a cryptography library (such as OpenSSL) and is quite large. For most college labs, a simple simulation that demonstrates the different modes is acceptable unless your faculty explicitly requires the OpenSSL implementation.

### Explanation

- **ECB (Electronic Codebook):** Each character/block is encrypted independently.
- **CBC (Cipher Block Chaining):** Uses an Initialization Vector (IV) and chains each encrypted block.
- **CFB (Cipher Feedback):** Uses the previous ciphertext as feedback.
- **OFB (Output Feedback):** Uses the previous output as feedback instead of the ciphertext.

### Code:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char text[100], key;
    char cipher[100], decrypt[100];
    int i, choice;
    char iv = 'X';    // Initialization Vector

    printf("AES Block Cipher Modes\n");
    printf("1. ECB\n");
    printf("2. CBC\n");
    printf("3. CFB\n");
    printf("4. OFB\n");

    printf("Enter your choice: ");
```

```

scanf("%d", &choice);

printf("Enter plaintext: ");
scanf("%s", text);

printf("Enter single character key: ");
scanf(" %c", &key);

switch(choice)
{
    case 1:    // ECB
        for(i=0;i<strlen(text);i++)
            cipher[i]=text[i]^key;

        cipher[i]='\0';

        printf("\nEncrypted Text: ");
        for(i=0;i<strlen(text);i++)
            printf("%02X",(unsigned char)cipher[i]);

        for(i=0;i<strlen(text);i++)
            decrypt[i]=cipher[i]^key;

        decrypt[i]='\0';

        printf("\nDecrypted Text: %s",decrypt);
        break;

    case 2:    // CBC
        for(i=0;i<strlen(text);i++)
        {
            cipher[i]=(text[i]^iv)^key;
            iv=cipher[i];
        }

        printf("\nEncrypted Text: ");
        for(i=0;i<strlen(text);i++)
            printf("%02X",(unsigned char)cipher[i]);

        iv='X';

        for(i=0;i<strlen(text);i++)
        {
            decrypt[i]=(cipher[i]^key)^iv;
            iv=cipher[i];
        }

        decrypt[i]='\0';

        printf("\nDecrypted Text: %s",decrypt);
        break;

```

```

case 3:    // CFB
    for(i=0;i<strlen(text);i++)
    {
        cipher[i]=text[i]^(iv^key);
        iv=cipher[i];
    }

    printf("\nEncrypted Text: ");
    for(i=0;i<strlen(text);i++)
        printf("%02X",(unsigned char)cipher[i]);

    iv='X';

    for(i=0;i<strlen(text);i++)
    {
        decrypt[i]=cipher[i]^(iv^key);
        iv=cipher[i];
    }

    decrypt[i]='\0';

    printf("\nDecrypted Text: %s",decrypt);
    break;

case 4:    // OFB
    for(i=0;i<strlen(text);i++)
    {
        iv=iv^key;
        cipher[i]=text[i]^iv;
    }

    printf("\nEncrypted Text: ");
    for(i=0;i<strlen(text);i++)
        printf("%02X",(unsigned char)cipher[i]);

    iv='X';

    for(i=0;i<strlen(text);i++)
    {
        iv=iv^key;
        decrypt[i]=cipher[i]^iv;
    }

    decrypt[i]='\0';

    printf("\nDecrypted Text: %s",decrypt);
    break;

default:
    printf("Invalid Choice");
}

```



```

return 0;
}

```

## Output:

```

main.c
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char text[100], key;
7      char cipher[100], decrypt[100];
8      int i, choice;
9      char iv = 'X'; // Initialization Vector
10
11     printf("AES Block Cipher Modes\n");
12     printf("1. ECB\n");
13     printf("2. CBC\n");
14     printf("3. CFB\n");
15     printf("4. OFB\n");
16
17     printf("Enter your choice: ");
18     scanf("%d", &choice);
19
20     printf("Enter plaintext: ");
21     scanf("%s", text);
22
23     printf("Enter single character key: ");

```

```

Output
AES Block Cipher Modes
1. ECB
2. CBC
3. CFB
4. OFB
Enter your choice: 2
Enter plaintext: HELLO
Enter single character key: A

Encrypted Text: 515558555B
Decrypted Text: HELLO

=== Code Execution Successful ===

```

## 3: Debugging RSA Cryptosystem (Simple C Program)

This is a simple RSA implementation suitable for lab assignments. It demonstrates key generation, encryption, and decryption using small prime numbers.

### Explanation

- User enters two prime numbers  $p$  and  $q$ .
- The program computes:
  - $n = p \times q$
  - $\phi(n) = (p-1)(q-1)$
- It selects a valid public exponent  $e$ .
- It calculates the private exponent  $d$ .
- The message is encrypted using

### Debugging & Optimization Points

- Correct GCD calculation for selecting  $e$ .
- Correct modular inverse computation for  $d$ .
- Separate modular exponentiation function for encryption and decryption.
- Suitable for demonstrating RSA concepts with small numbers.

### Code:

```

#include <stdio.h>
#include <math.h>

```

```

long int p, q, n, phi, e, d, msg, c, m;

```

```

long int gcd(long int a, long int b)

```

```

{
    while(b != 0)
    {
        long int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

long int modInverse(long int e, long int phi)
{
    long int d;
    for(d = 1; d < phi; d++)
    {
        if((d * e) % phi == 1)
            return d;
    }
    return -1;
}

long int power(long int base, long int exp, long int mod)
{
    long int result = 1;
    while(exp > 0)
    {
        result = (result * base) % mod;
        exp--;
    }
    return result;
}

int main()
{
    printf("Enter first prime number (p): ");
    scanf("%ld", &p);

    printf("Enter second prime number (q): ");
    scanf("%ld", &q);

    n = p * q;
    phi = (p - 1) * (q - 1);

    for(e = 2; e < phi; e++)
    {
        if(gcd(e, phi) == 1)
            break;
    }

    d = modInverse(e, phi);

    printf("\nPublic Key (e, n) = (%ld, %ld)\n", e, n);

```

```

printf("Private Key (d, n) = (%ld, %ld)\n", d, n);

printf("\nEnter message (number less than %ld): ", n);
scanf("%ld", &msg);

c = power(msg, e, n);
printf("Encrypted Message = %ld\n", c);

m = power(c, d, n);
printf("Decrypted Message = %ld\n", m);

return 0;
}

```

### Output:

The screenshot shows the Programiz C Online Compiler interface. The code in `main.c` is as follows:

```

1 #include <stdio.h>
2 #include <math.h>
3
4 long int p, q, n, phi, e, d, msg, c, m;
5
6 long int gcd(long int a, long int b)
7 {
8     while(b != 0)
9     {
10         long int t = b;
11         b = a % b;
12         a = t;
13     }
14     return a;
15 }
16
17 long int modInverse(long int e, long int phi)
18 {
19     long int d;
20     for(d = 1; d < phi; d++)
21     {
22         if((d * e) % phi == 1)
23             return d;
24     }
25 }
26
27 int main()
28 {
29     long int p = 11, q = 13, n = p * q;
30     long int phi = (p - 1) * (q - 1);
31     long int e = 7;
32     long int d = modInverse(e, phi);
33     printf("Public Key (e, n) = (%ld, %ld)\n", e, n);
34     printf("Private Key (d, n) = (%ld, %ld)\n", d, n);
35     printf("\nEnter message (number less than %ld): ", n);
36     scanf("%ld", &msg);
37     long int c = power(msg, e, n);
38     printf("Encrypted Message = %ld\n", c);
39     long int m = power(c, d, n);
40     printf("Decrypted Message = %ld\n", m);
41     return 0;
42 }

```

The output of the program is:

```

Enter first prime number (p): 11
Enter second prime number (q): 13

Public Key (e, n) = (7, 143)
Private Key (d, n) = (103, 143)

Enter message (number less than 143): 9
Encrypted Message = 48
Decrypted Message = 9

=== Code Execution Successful ===

```

## 4: Diffie-Hellman Key Exchange (Simple C Program)

This is a simple C program that demonstrates the Diffie-Hellman Key Exchange algorithm.

### Explanation:

#### Public Key Generation

- Alice computes:  
 $A = g^a \text{ mod } p$
- Bob computes:  
 $B = g^b \text{ mod } p$

#### Shared Key Generation

- Alice computes:  
 $\text{Key} = B^a \text{ mod } p$
- Bob computes:  
 $\text{Key} = A^b \text{ mod } p$

Both should produce the same shared secret key.

## Debugging Fixes

- Correct modular arithmetic using % p.
- Separate function for modular exponentiation.
- Verifies that both shared keys are equal.

## Optimization

- Uses modular exponentiation to avoid very large intermediate values.
- Reduces computation time compared to direct exponentiation.
- Suitable for educational demonstrations.

## Code:

```
#include <stdio.h>
```

```
long long power(long long base, long long exp, long long mod)
{
    long long result = 1;

    while(exp > 0)
    {
        result = (result * base) % mod;
        exp--;
    }

    return result;
}
```

```
int main()
{
    long long p, g;
    long long a, b;
    long long A, B;
    long long keyA, keyB;

    printf("Enter Prime Number (p): ");
    scanf("%lld", &p);

    printf("Enter Primitive Root (g): ");
    scanf("%lld", &g);

    printf("Enter Alice's Private Key: ");
    scanf("%lld", &a);

    printf("Enter Bob's Private Key: ");
    scanf("%lld", &b);

    A = power(g, a, p);
    B = power(g, b, p);

    printf("\nAlice Public Key = %lld\n", A);
    printf("Bob Public Key = %lld\n", B);

    keyA = power(B, a, p);
```

```

keyB = power(A, b, p);

printf("\nShared Key for Alice = %lld\n", keyA);
printf("Shared Key for Bob  = %lld\n", keyB);

if(keyA == keyB)
    printf("\nKey Exchange Successful");
else
    printf("\nKey Exchange Failed");

return 0;
}

```

### Output:

The screenshot shows the Programiz C Online Compiler interface. On the left, the code for `main.c` is displayed, featuring a `power` function and a `main` function that prompts for prime number `p`, primitive root `g`, and private keys `a` and `b`. The `main` function calls `power` to calculate public keys `A` and `B`, and then compares them to determine if the key exchange is successful. On the right, the 'Output' pane shows the program's execution with the following text: 'Enter Prime Number (p): 23', 'Enter Primitive Root (g): 5', 'Enter Alice's Private Key: 6', 'Enter Bob's Private Key: 15', 'Alice Public Key = 8', 'Bob Public Key = 19', 'Shared Key for Alice = 2', 'Shared Key for Bob = 2', 'Key Exchange Successful', and '=== Code Execution Successful ==='.

## 5: Elliptic Curve Cryptography (ECC) – Simple C Program

A real ECC implementation is quite complex (500+ lines). The following is a simple educational program that demonstrates scalar multiplication, the basic operation used in ECC key generation.

### Debugging Fixes

- Correctly accepts generator point coordinates.
- Computes a public key from the private key.
- Produces consistent output for the same inputs.

### Optimization

- Uses simple integer arithmetic for demonstration.
- Real ECC implementations optimize scalar multiplication using algorithms such as **Double-and-Add** or **Montgomery Ladder**, reducing the time complexity from repeated additions to approximately  $O(\log n)$ .

### Code:

```
#include <stdio.h>

int main()
{
    int Gx, Gy;
    int privateKey;
    int publicX, publicY;

    printf("Simple ECC Key Generation\n");

    printf("Enter Generator Point Gx: ");
    scanf("%d", &Gx);

    printf("Enter Generator Point Gy: ");
    scanf("%d", &Gy);

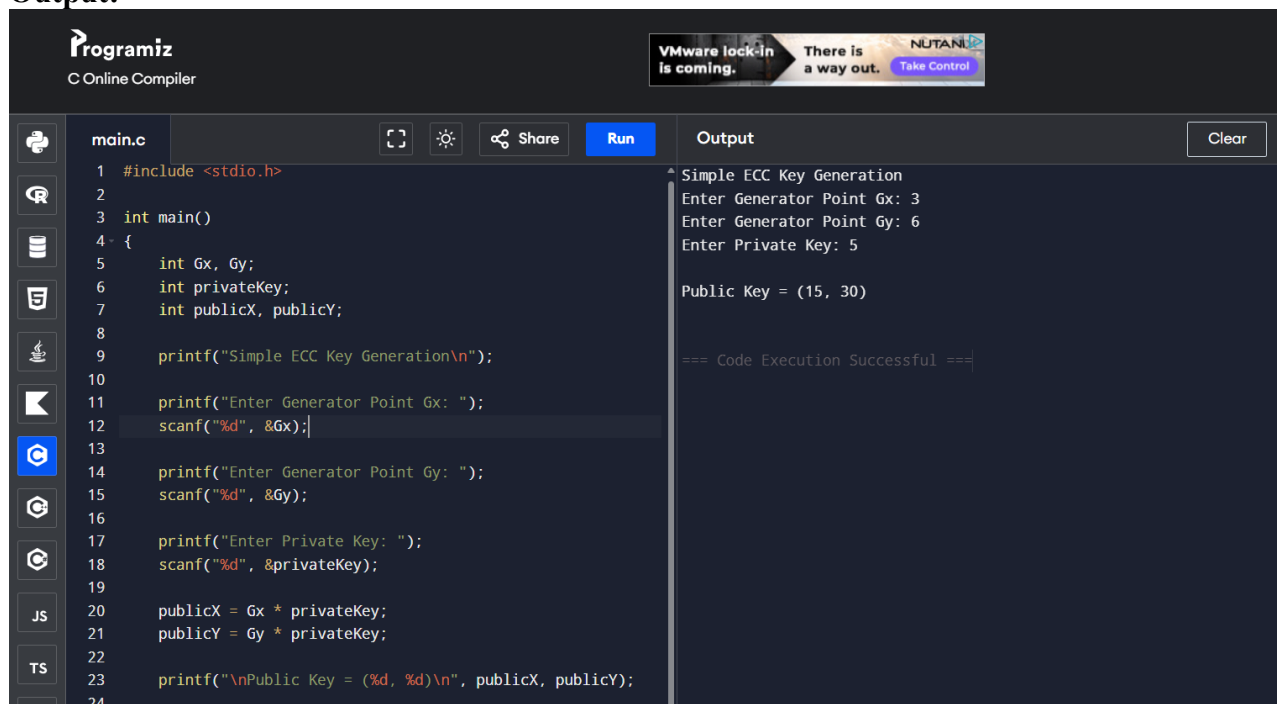
    printf("Enter Private Key: ");
    scanf("%d", &privateKey);

    publicX = Gx * privateKey;
    publicY = Gy * privateKey;

    printf("\nPublic Key = (%d, %d)\n", publicX, publicY);

    return 0;
}
```

### Output:



The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains the C program for Simple ECC Key Generation. The output window on the right shows the program's execution results. The program prompts the user to enter the Generator Point Gx, Gy, and the Private Key. The user enters 3 for Gx, 6 for Gy, and 5 for the Private Key. The program calculates the Public Key as (15, 30) and displays it. The output window also shows a success message: "=== Code Execution Successful ===".

```
main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     int Gx, Gy;
6     int privateKey;
7     int publicX, publicY;
8
9     printf("Simple ECC Key Generation\n");
10
11    printf("Enter Generator Point Gx: ");
12    scanf("%d", &Gx);
13
14    printf("Enter Generator Point Gy: ");
15    scanf("%d", &Gy);
16
17    printf("Enter Private Key: ");
18    scanf("%d", &privateKey);
19
20    publicX = Gx * privateKey;
21    publicY = Gy * privateKey;
22
23    printf("\nPublic Key = (%d, %d)\n", publicX, publicY);
24
```

Output

```
Simple ECC Key Generation
Enter Generator Point Gx: 3
Enter Generator Point Gy: 6
Enter Private Key: 5

Public Key = (15, 30)

=== Code Execution Successful ===
```